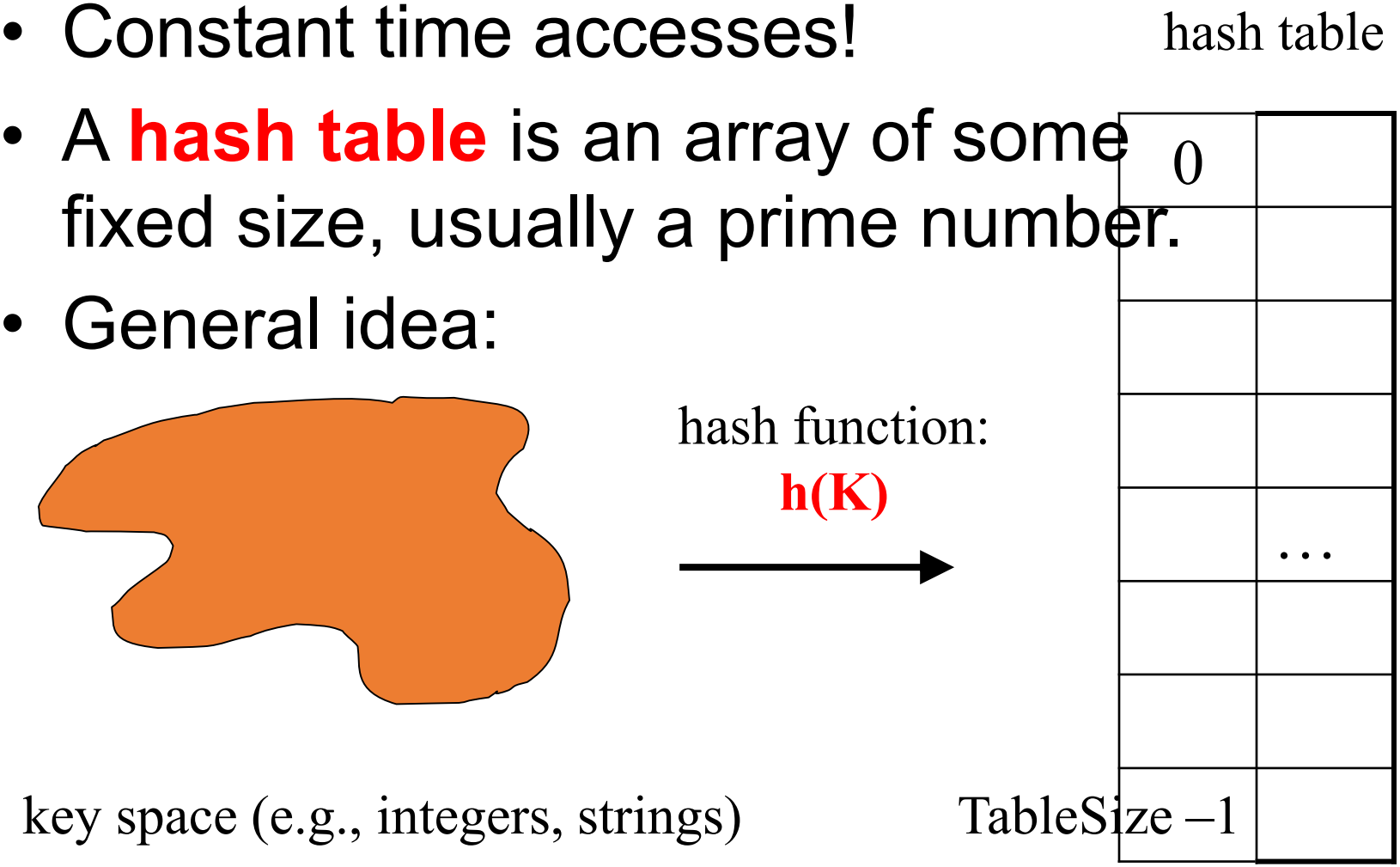
Hash Tables

- Constant time accesses!
- A **hash table** is an array of some fixed size, usually a prime number.
- General idea:



Example

- key space = integers
- TableSize = 10

- **h**(K) = K mod 10

- **Insert**: 7, 18, 41, 94

0	
1	41
2	
3	
4	94
5	
6	
7	7
8	18
9	

Another Example

- key space = integers
- TableSize = 6

- **h**(K) = K mod 6

- **Insert**: 7, 18, 41, 34

0	18
1	7
2	
3	
4	34
5	41

- Definition of a Hash Table:
  - A hash table is a data structure that organizes data using a hash function. It transforms keys into indices within an array, allowing for quick data retrieval and storage.
- Purpose: Efficient Data Retrieval and Storage:
  - The primary purpose of a hash table is to provide an efficient mechanism for storing and retrieving data.
- Efficiency:
  - Fast access to data compared to linear data structures like arrays or linked lists.
  - Constant time complexity for basic operations on average, making it suitable for large datasets.

Components: Key, Hash Function, Bucket:

- Key:
  - Represents the unique identifier for each piece of data stored in the hash table.
  - Example: In a student database, the student ID could be the key.
- Hash Function:
  - An algorithm that takes a key as input and produces a fixed-size string of characters, which is used as the index in the array. Should distribute keys uniformly across the array to minimize collisions.
  - Example: A simple hash function might sum the ASCII values of characters in the key.
- Bucket:
  - Each index or slot in the hash table array is called a bucket.
  - A bucket can store multiple key-value pairs, handling cases where different keys hash to the same index (collisions).
  - Example: If two students have the same hash index, their records are stored in the same bucket.

Definition of a Hash Function:

- A hash function is a mathematical function that takes an input (a key) and produces a fixed-size string of characters, which is used as an index or address in the hash table.
- Purpose: Map Keys to Indices in the Hash Table:
- The primary purpose of a hash function is to efficiently map keys to indices within the hash table.
- This mapping allows for the quick storage and retrieval of data, providing constant-time complexity on average.

Characteristics of a Good Hash Function:

1. Uniformity:

- A good hash function should distribute keys uniformly across the indices of the hash table.
- This helps minimize collisions and ensures that each bucket has a roughly equal number of keys.
- Example: If a hash function always produces the same index for a certain set of keys, it leads to poor uniformity.

2. Efficiency:

- The hash function should be computationally efficient to calculate, ensuring quick access to data.
- Avoid complex operations that might slow down the hashing process.
- Example: A hash function that involves complex mathematical computations may be less efficient than a simpler one.

3. Deterministic:

- Given the same input (key), a hash function should always produce the same output (hash code).
- Deterministic behaviour ensures consistency in indexing.
- Example: If hashing the same key multiple times produces different indices, it leads to unpredictability.

Hash Functions

1. **simple/fast** to compute,
2. Avoid **collisions**
3. have keys distributed **evenly** among cells.

Perfect Hash function:

Sample Hash Functions:

- key space = strings
- $s = s_0\ s_1\ s_2 \ldots s_{k-1}$

1. $h(s) = s_0 \bmod \text{TableSize}$
2. $h(s) = \left(\sum_{i=0}^{k-1} s_i \right) \bmod \text{TableSize}$
2. $h(s) = \left(\sum_{i=0}^{k-1} s_i \cdot 37^i \right) \bmod \text{TableSize}$

Methods for Hash Key Generation (Different Types of Hash)

- Division Method
- Multiplication Method
- Extraction Method
- Mid Square Method
- Folding
- Universal Method

Division Method

- The division method is a simple and commonly used technique for designing hash functions. It involves dividing the key by the size of the hash table and using the remainder as the index. The formula for the division method is:

$$[\ h(k) = k \bmod m\ ]$$

where:

- (h(k) is the hash value for key (k)
- (k) is the key being hashed,
- (m) is the size of the hash table.

- Steps to calculate hask Keys

1. Choose the Size of the Hash Table (m):
   - The size of the hash table (m) is a crucial factor. It should ideally be a prime number to minimize collisions and ensure a more uniform distribution of keys.
2. Calculate the Hash Value:
   - Take the key (\(k \)) and compute \(k \mod m \) to get the hash value.
   - This operation effectively maps the key to an index in the hash table.

- Advantages:
  - Simple and easy to implement.
  - Efficient for integer keys.

Multiplication Method

- The multiplication method is another technique for designing hash functions. It involves multiplying the key by a constant fraction and extracting the fractional part of the product to determine the index in the hash table. The formula for the multiplication method is:

$$[\ h(k) = m\ .(k\ .A \bmod 1)\ ]$$

where:

- (h(k) is the hash value for key (k),
- (k) is the key being hashed,
- (m) is the size of the hash table,
- (A) is a constant, (0 < A < 1).

Steps :

1. Choose the Size of the Hash Table (m)
   - Similar to the division method, the size of the hash table (m) should ideally be a prime number.
2. Choose a Constant (A):
   - Select a constant (A) such that (0 < A < 1).
3. Calculate the Hash Value:
   - Compute (m .(k . A mod 1)) to get the hash value.
   - The multiplication operation helps distribute keys more uniformly across the hash table.

Example: A hash table with a size of 11 (m = 11) and (A = 0.6180339887. If your key is 42, the hash value would be (11.(42 .0.6180339887 mod 1)).

- Advantages:
  - Offers a good distribution of keys when properly chosen.
  - Effective for floating-point keys.

Extraction Method

- The extraction method is a hash function technique used for keys that consist of multiple components or fields. In this method, certain components of the key are selected and combined to form a composite key. This composite key is then hashed to determine the index in the hash table. The extraction method is particularly useful when dealing with complex keys.

  The formula for the extraction method is:

$$[h(k) = g(k_1, k_2, ..., k_n)]$$

where:

- (h(k)) is the hash value for key (k),
- (k_1, k_2, ..., k_n) are components or fields of the key (k),
- g() is a combining function.

steps involved:

1. hoose Key Components:
   - Identify the specific components or fields of the key that will be used for hashing. These components can be chosen based on the nature of the data and the desired characteristics of the hash function.
2. Combining Function:
   - Define a combining function g() that takes the selected components and combines them into a single composite key.
   - The combining function should produce a unique composite key for different combinations of the selected components.
3. Calculate the Hash Value:
   - Apply the hash function to the composite key to determine the index in the hash table.
4. Example:
   - Suppose you have a composite key (k_1, k_2) , where (k_1) is a string representing a username and (k_2) is an integer representing a user ID. The hash function could be (h(k) = text{length}(k_1) + k_2).

- Advantages:
  - Allows flexibility in handling complex keys with multiple components.
  - Enables customization based on the specific characteristics of the data.

- Example Code (Python):

```
def extraction_hash(username, user_id):
    return len(username) + user_id

# Example Usage
username = "john_doe"
user_id = 123
hash_value = extraction_hash(username, user_id)

print(f"The hash value for key ({username}, {user_id}) is
{hash_value}.")
```

Mid Square :

The Mid Square Method is a simple technique for designing hash functions, especially for integer keys. It involves squaring the key, extracting a portion of the result, and using that portion as the hash value. The basic formula for the Mid Square Method is:

[h(k) = text{extract}((k^2), w)]

where:
- (h(k) \) is the hash value for key \(k \),
- \(\text{extract}() \) is a function that extracts a portion of the result,
- \(w \) is the width of the extracted portion.

the steps involved:
1. quare the Key:
   - Square the key (k) to get (k^2).

2. Extract a Portion:
   - Use the (text{extract}()) function to extract a portion of the result. This portion becomes the hash value.
   - The text{extract}()) function typically involves taking the middle digits of the squared result.
3. Calculate the Hash Value:
   - Apply the hash function to determine the index in the hash table.
4. Example:
   - Suppose you have a key (k = 427\). The square is (k^2 = 182329). If you extract the middle two digits, the hash value would be (23).

Advantages:
- Simple and easy to implement.
- Suitable for integer keys.

Collision

❖ Definition: A collision occurs in a hash table when two or more keys hash to the same index (bucket). Since each index can only store one key-value pair, collisions need to be addressed to avoid data loss and ensure proper retrieval.

Types: Open Addressing and Closed Addressing:

❖ Open Addressing:

- ✓ In open addressing, when a collision occurs, the algorithm looks for the next available slot in the hash table until an empty slot is found.
- ✓ Types of open addressing include linear probing, quadratic probing, and double hashing.
- ✓ Example: If index `i` is occupied, try index `i + 1`, `i + 2`, and so on, until an empty slot is found.

❖ Closed Addressing (Chaining):

- ✓ In closed addressing, each bucket in the hash table is a linked list. Colliding keys are stored in the same bucket as a linked list.
- ✓ Example: If two keys hash to the same index, they are stored as nodes in a linked list at that index.

Why Collision Occur

1. Limited Hash Table Size:

   ✓ If the hash table size is not large enough relative to the number of keys, collisions are more likely.

2. Hash Function Imperfections:

   ✓ Even with a good hash function, it may not be possible to avoid all collisions, especially as the number of keys increases.

3. Limited Range of Hash Values:

   ✓ If the hash function produces a limited range of values, collisions are more probable.

4. Clustering:

   ✓ Some patterns in the data or the hash function may cause keys to cluster around certain indices, leading to collisions.

Collision Resolution

Two ways to resolve collisions:

1. Separate Chaining
2. Open Addressing (linear probing, quadratic probing, double hashing)

Separate Chaining

0	10
1	
2	22/12/42
3	
4	
5	
6	
7	107
8	
9	

Insert:
10
22
107
12
42

- **<u>Separate chaining</u>**: All keys that map to the same hash value are kept in a list (or "bucket").

Analysis of find

- Defn: The load factor, λ, of a hash table is the ratio:

$$\frac{N}{M} \quad \begin{array}{l} \leftarrow \text{no. of elements} \\ \leftarrow \text{table size} \end{array}$$

For separate chaining, λ = average # of elements in a bucket

- Unsuccessful find:

- Successful find:

How big should the hash table be?

- For Separate Chaining:

tableSize: Why Prime?

- Suppose
  - data stored in hash table: 7160, 493, 60, 55, 321, 900, 810
  - tableSize = 10

    data hashes to 0, 3, <u>0</u>, 5, 1, <u>0</u>, <u>0</u>
  - tableSize = 11

    data hashes to 10, 9, 5, 0, 2, <u>9</u>, 7

Real-life data tends to have a pattern

Being a multiple of 11 is usually *not* the pattern ☺

Open Addressing

0	8
1	109
2	10
3	
4	
5	
6	
7	
8	38
9	19

Insert:
38
19
8
109
10

- **<u>Linear Probing</u>**: after checking spot h(k), try spot h(k)+1, if that is full, try h(k)+2, then h(k)+3, etc.

Terminology Alert!

"**Open** Hashing"

equals

"Separate Chaining"

"Closed Hashing"

equals

"**Open** Addressing"

Linear Probing

$$f(i) = i$$

- Probe sequence:

  0th probe = h(k) mod TableSize

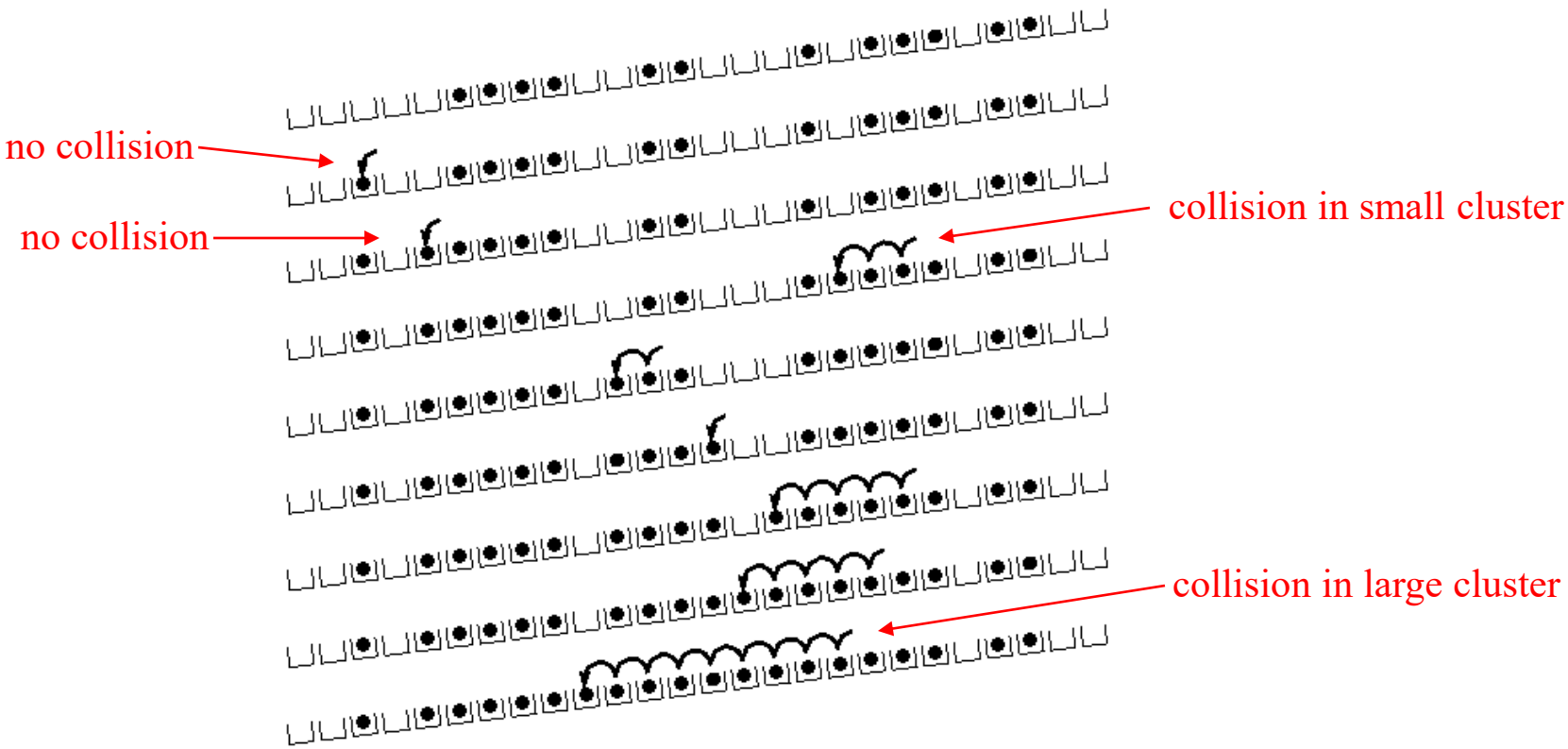
  1th probe = (h(k) + 1) mod TableSize

  2th probe = (h(k) + 2) mod TableSize

  . . .

  i^{th} probe = (h(k) + i) mod TableSize

Linear Probing – Clustering



[R. Sedgewick]

Load Factor in Linear Probing

- For *any* $\lambda < 1$, linear probing *will* find an empty slot
- Expected # of probes (for large table sizes)
  - successful search: $\frac{1}{2}\left(1+\frac{1}{(1-\lambda)}\right)$
  - unsuccessful search: $\frac{1}{2}\left(1+\frac{1}{(1-\lambda)^2}\right)$
- Linear probing suffers from ***primary clustering***
- Performance quickly degrades for $\lambda > 1/2$

Quadratic Probing

Less likely to encounter Primary Clustering

$f(i) = i^2$

- Probe sequence:

  0th probe = h(k) mod TableSize

  1th probe = (h(k) + 1) mod TableSize

  2th probe = (h(k) + 4) mod TableSize

  3th probe = (h(k) + 9) mod TableSize

  . . .

  i^{th} probe = $(h(k) + i^2)$ mod TableSize

Quadratic Probing

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Insert:
89
18
49
58
79

Quadratic Probing Example

insert(76)
76%7 = 6

insert(40)
40%7 = 5

insert(48)
48%7 = 6

insert(5)
5%7 = 5

insert(55)
55%7 = 6

0	
1	
2	
3	
4	
5	
6	76

But… insert(47)
47%7 = 5

Quadratic Probing: Properties

- For *any* $\lambda < ½$, quadratic probing will find an empty slot; for bigger λ, quadratic probing *may* find a slot
- Quadratic probing does not suffer from *primary* clustering: keys hashing to the same *area* are not bad
- But what about keys that hash to the same *spot*?
  - ***Secondary Clustering!***

Double Hashing

$f(i) = i * g(k)$

where g is a second hash function

- Probe sequence:

  0th probe = h(k) mod TableSize

  1th probe = (h(k) + g(k)) mod TableSize

  2th probe = (h(k) + 2*g(k)) mod TableSize

  3th probe = (h(k) + 3*g(k)) mod TableSize

  . . .

  i^{th} probe = (h(k) + i*g(k)) mod TableSize

Double Hashing Example

$h(k) = k \bmod 7$ and $g(k) = 5 - (k \bmod 5)$

	76	93	40	47	10	55
0						
1				47	47	47
2		93	93	93	93	93
3					10	10
4						55
5			40	40	40	40
6	76	76	76	76	76	76
Probes	1	1	1	2	1	2

Resolving Collisions with Double Hashing

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Hash Functions:

$H(K) = K \bmod M$

$H_2(K) = 1 + ((K/M) \bmod (M-1))$

$M =$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

147

43

Dynamic Hashing

- For good performance , it is necessary **to increase size of hash** table; whenever its loading density exceeds a pre specified threshold

EX: if there are b buckets in hash table and using a division hash function then **D=b.**

When an insert causes the loading density to exceed the prespecified threshold. We use array doubling to increase the number of buckets to **2b+1**. This change in divisor require to rebuild hash table

- **Dynamic Hashing** : Also known as extendible hashing , aims to reduce the rebuild time by ensuring that each rebuild changes the home bucket for the entries in only 1 bucket.
- **Objective of dynamic** hashing is to provide acceptable hash table performance on a per operation basis.

Types of Dynamic Hashing

1. Using Directory
2. With out directory

Both forms, hash function h used to map key into non negative integers.

Ex: h(k) transforms keys into 6 bit non negative integers.

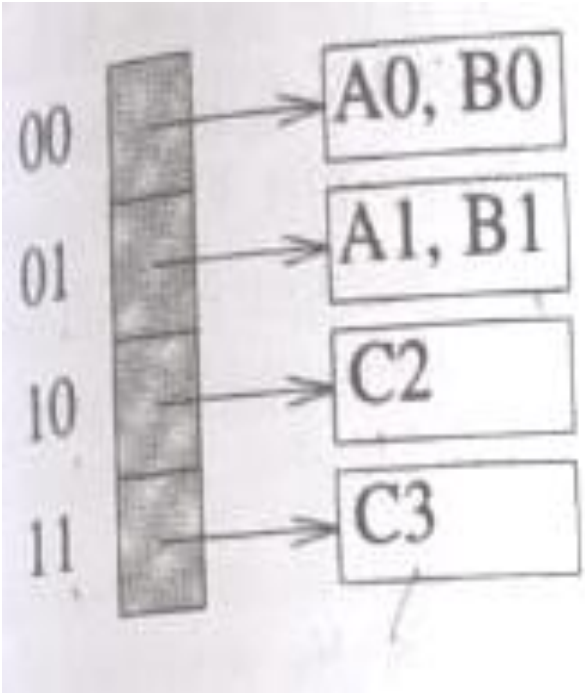
Let keys with 2 character each and h transfroms letters such as A B and C into bit sequence like 100, 101 and 110

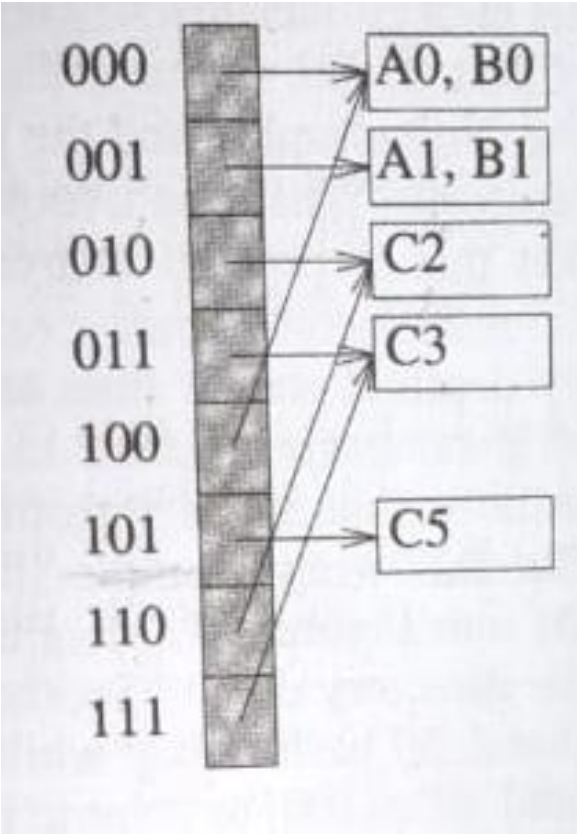
Digits 0 to 7 are transformed into their 3 bit representation

K	h(K)
A0	100 000
A1	100 001
B0	101 000
B1	101 001
C1	110 001
C2	110 010
C3	110 011
C5	110 101

Dynamic Hash Table with directories

The Number of bits of h(k) used to index the directory is called the **directory depth**





Depth = 2

Depth = 3